

Langages 2

N3 informatique, S5 – CM1

2026-2027

intervenants :

- ▶ Bruno GUILLON (responsable du cours, CM et TD)
- ▶ Olivier RAYNAUD (TD)

évaluation : 100% contrôle continu

- ▶ présence en TD/TP (3 absences $\implies$ défaillances)
- ▶ une épreuve intermédiaire (peut-être un QCM)
- ▶ un contrôle final (écrit)

contexte :

- ▶ suite de *“Introduction à la théorie des langages”* (N2)
- ▶ préparation de *“Compilation”* (N3, S5, même UE)

Rappels & notations

(langages réguliers)

Rappels & notations : alphabet, mots, langages

alphabet : un ensemble fini, non-vidé (souvent noté Σ , Γ , Δ , ...) dont les éléments sont appelés *lettres* (ou *symboles*)

Rappels & notations : alphabet, mots, langages

alphabet : un ensemble fini, non-vide (souvent noté Σ , Γ , Δ , ...) ;
dont les éléments sont appelés *lettres* (ou *symboles*)

mot : une suite finie de lettres

base : la suite vide est un mot (appelé *mot vide* et notée ε) ;

règle : si u est un mot et a est une lettre, alors $a \cdot u$ est un mot.

on note Σ^* l'ensemble des mots sur Σ ;
on ne différencie pas une lettre a du mot a .

Rappels & notations : alphabet, mots, langages

alphabet : un ensemble fini, non-vidé (souvent noté Σ , Γ , Δ , ...) dont les éléments sont appelés *lettres* (ou *symboles*)

mot : une suite finie de lettres

base : la suite vide est un mot (appelé *mot vide* et notée ε) ;

règle : si u est un mot et a est une lettre, alors $a \cdot u$ est un mot.

on note Σ^* l'ensemble des mots sur Σ ;

on ne différencie pas une lettre a du mot a .

longueur : le nombre de lettres d'un mot u , noté $|u|$

base : $|\varepsilon| = 0$; **règle** : $|a \cdot u| = 1 + |u|$.

Rappels & notations : alphabet, mots, langages

alphabet : un ensemble fini, non-vidé (souvent noté Σ , Γ , Δ , ...) dont les éléments sont appelés *lettres* (ou *symboles*)

mot : une suite finie de lettres

base : la suite vide est un mot (appelé *mot vide* et notée ε) ;

règle : si u est un mot et a est une lettre, alors $a \cdot u$ est un mot.

on note Σ^* l'ensemble des mots sur Σ ;

on ne différencie pas une lettre a du mot a .

longueur : le nombre de lettres d'un mot u , noté $|u|$

base : $|\varepsilon| = 0$; **règle** : $|a \cdot u| = 1 + |u|$.

$|u|_a$: pour $a \in \Sigma$, le nombre d'occurrence de a dans u .

base : $|\varepsilon|_a = 0$; **règles** : $|a \cdot u|_a = 1 + |u|$;
 $|b \cdot u|_a = |u|$ pour $b \neq a$.

Rapports & notations : alphabet, mots, langages

alphabet : un ensemble fini, non-vidé (souvent noté Σ , Γ , Δ , ...) dont les éléments sont appelés *lettres* (ou *symboles*)

mot : une suite finie de lettres

base : la suite vide est un mot (appelé *mot vide* et notée ε) ;

règle : si u est un mot et a est une lettre, alors $a \cdot u$ est un mot.

on note Σ^* l'ensemble des mots sur Σ ;

on ne différencie pas une lettre a du mot a .

longueur : le nombre de lettres d'un mot u , noté $|u|$

base : $|\varepsilon| = 0$; **règle** : $|a \cdot u| = 1 + |u|$.

$|u|_a$: pour $a \in \Sigma$, le nombre d'occurrence de a dans u .

base : $|\varepsilon|_a = 0$; **règles** : $|a \cdot u|_a = 1 + |u|$;
 $|b \cdot u|_a = |u|$ pour $b \neq a$.

langage : un ensemble (fini ou infini) de mots

i.e., un sous-ensemble de Σ^*

Exemples : $\emptyset$, $\{\varepsilon\}$, $\{a, ba\}$, $\{b, ba, baa, \dots\}$, ...

Rappels & notations : opérations sur les mots

concaténation : $\Sigma^* \times \Sigma^* \rightarrow \Sigma^*$, notée $u \cdot v$ ou uv

base : $\varepsilon \cdot v = v$; **règle** : $(a \cdot u) \cdot v = a \cdot (u \cdot v)$.

Rappels & notations : opérations sur les mots

concaténation : $\Sigma^* \times \Sigma^* \rightarrow \Sigma^*$, notée $u \cdot v$ ou uv

base : $\varepsilon \cdot v = v$; **règle** : $(a \cdot u) \cdot v = a \cdot (u \cdot v)$.

puissance : $\Sigma^* \times \mathbb{N} \rightarrow \Sigma^*$, notée u^i

base : $u^0 = \varepsilon$; **règle** : $u^{i+1} = u \cdot u^i$.

Rappels & notations : opérations sur les mots

concaténation : $\Sigma^* \times \Sigma^* \rightarrow \Sigma^*$, notée $u \cdot v$ ou uv

base : $\varepsilon \cdot v = v$; **règle** : $(a \cdot u) \cdot v = a \cdot (u \cdot v)$.

puissance : $\Sigma^* \times \mathbb{N} \rightarrow \Sigma^*$, notée u^i

base : $u^0 = \varepsilon$; **règle** : $u^{i+1} = u \cdot u^i$.

miroir : $\Sigma^* \rightarrow \Sigma^*$, notée $\overleftarrow{u}$

base : $\overleftarrow{\varepsilon} = \varepsilon$; **règle** : $\overleftarrow{a \cdot u} = \overleftarrow{u} \cdot a$.

Rappels & notations : opérations sur les mots

concaténation : $\Sigma^* \times \Sigma^* \rightarrow \Sigma^*$, notée $u \cdot v$ ou uv

base : $\varepsilon \cdot v = v$; **règle** : $(a \cdot u) \cdot v = a \cdot (u \cdot v)$.

puissance : $\Sigma^* \times \mathbb{N} \rightarrow \Sigma^*$, notée u^i

base : $u^0 = \varepsilon$; **règle** : $u^{i+1} = u \cdot u^i$.

miroir : $\Sigma^* \rightarrow \Sigma^*$, notée $\overleftarrow{u}$

base : $\overleftarrow{\varepsilon} = \varepsilon$; **règle** : $\overleftarrow{a \cdot u} = \overleftarrow{u} \cdot a$.

Exercices

- ▶ Montrer que ε est un élément neutre pour la concaténation.
- ▶ Montrer par induction que la concaténation est associative.
- ▶ Montrer que la concaténation n'est pas commutative.
- ▶ Montrer par récurrence que $\varepsilon^i = \varepsilon$ pour tout i .
- ▶ Montrer par induction que le miroir préserve les longueurs.
- ▶ Montrer par induction que $\overleftarrow{u \cdot v} = \overleftarrow{v} \cdot \overleftarrow{u}$.

Rappels & notations : opérations sur les mots

concaténation : $\Sigma^* \times \Sigma^* \rightarrow \Sigma^*$, notée $u \cdot v$ ou uv

base : $\varepsilon \cdot v = v$; **règle** : $(a \cdot u) \cdot v = a \cdot (u \cdot v)$.

puissance : $\Sigma^* \times \mathbb{N} \rightarrow \Sigma^*$, notée u^i

base : $u^0 = \varepsilon$; **règle** : $u^{i+1} = u \cdot u^i$.

miroir : $\Sigma^* \rightarrow \Sigma^*$, notée $\overleftarrow{u}$

base : $\overleftarrow{\varepsilon} = \varepsilon$; **règle** : $\overleftarrow{a \cdot u} = \overleftarrow{u} \cdot a$.

Exercices

- ▶ Montrer que ε est un **élément neutre** pour la concaténation.
- ▶ Montrer par induction que la concaténation est **associative**.
- ▶ Montrer que la concaténation n'est pas commutative.
- ▶ Montrer par récurrence que $\varepsilon^i = \varepsilon$ pour tout i .
- ▶ Montrer par induction que le miroir préserve les longueurs.
- ▶ Montrer par induction que $\overleftarrow{u \cdot v} = \overleftarrow{v} \cdot \overleftarrow{u}$.

Rappels & notations : opérations sur les langages

Rappels & notations : opérations sur les langages

concaténation : $\mathcal{P}(\Sigma^*) \times \mathcal{P}(\Sigma^*) \rightarrow \mathcal{P}(\Sigma^*)$, notée $L \cdot L'$

$$L \cdot L' = \{u \cdot v \mid u \in L, v \in L'\}$$

miroir : $\mathcal{P}(\Sigma^*) \rightarrow \mathcal{P}(\Sigma^*)$, notée $\overleftarrow{L}$

$$\overleftarrow{L} = \{\overleftarrow{u} \mid u \in L\}$$

Rappels & notations : opérations sur les langages

concaténation : $\mathcal{P}(\Sigma^*) \times \mathcal{P}(\Sigma^*) \rightarrow \mathcal{P}(\Sigma^*)$, notée $L \cdot L'$

$$L \cdot L' = \{u \cdot v \mid u \in L, v \in L'\}$$

puissance : $\mathcal{P}(\Sigma^*) \times \mathbb{N} \rightarrow \mathcal{P}(\Sigma^*)$, notée L^i

$$\text{base} : L^0 = \{\varepsilon\}; \quad \text{r\`egle} : L^{i+1} = L \cdot L^i.$$

miroir : $\mathcal{P}(\Sigma^*) \rightarrow \mathcal{P}(\Sigma^*)$, notée $\overleftarrow{L}$

$$\overleftarrow{L} = \{\overleftarrow{u} \mid u \in L\}$$

Rappels & notations : opérations sur les langages

concaténation : $\mathcal{P}(\Sigma^*) \times \mathcal{P}(\Sigma^*) \rightarrow \mathcal{P}(\Sigma^*)$, notée $L \cdot L'$

$$L \cdot L' = \{u \cdot v \mid u \in L, v \in L'\}$$

puissance : $\mathcal{P}(\Sigma^*) \times \mathbb{N} \rightarrow \mathcal{P}(\Sigma^*)$, notée L^i

base : $L^0 = \{\varepsilon\}$; **règle** : $L^{i+1} = L \cdot L^i$.

en général $L^i \neq \{w^i \mid w \in L\}$

miroir : $\mathcal{P}(\Sigma^*) \rightarrow \mathcal{P}(\Sigma^*)$, notée $\overleftarrow{L}$

$$\overleftarrow{L} = \{\overleftarrow{u} \mid u \in L\}$$

Rappels & notations : opérations sur les langages

concaténation : $\mathcal{P}(\Sigma^*) \times \mathcal{P}(\Sigma^*) \rightarrow \mathcal{P}(\Sigma^*)$, notée $L \cdot L'$

$$L \cdot L' = \{u \cdot v \mid u \in L, v \in L'\}$$

puissance : $\mathcal{P}(\Sigma^*) \times \mathbb{N} \rightarrow \mathcal{P}(\Sigma^*)$, notée L^i

base : $L^0 = \{\varepsilon\}$; **règle** : $L^{i+1} = L \cdot L^i$.

en général $L^i \neq \{w^i \mid w \in L\}$

miroir : $\mathcal{P}(\Sigma^*) \rightarrow \mathcal{P}(\Sigma^*)$, notée $\overleftarrow{L}$

$$\overleftarrow{L} = \{\overleftarrow{u} \mid u \in L\}$$

union, intersection, complémentation, différence, ... :

notés $L \cup L'$, $L \cap L'$, $\overline{L}$, $L \setminus L'$, ...

Rappels & notations : opérations sur les langages

concaténation : $\mathcal{P}(\Sigma^*) \times \mathcal{P}(\Sigma^*) \rightarrow \mathcal{P}(\Sigma^*)$, notée $L \cdot L'$

$$L \cdot L' = \{u \cdot v \mid u \in L, v \in L'\}$$

puissance : $\mathcal{P}(\Sigma^*) \times \mathbb{N} \rightarrow \mathcal{P}(\Sigma^*)$, notée L^i

base : $L^0 = \{\varepsilon\}$; **règle** : $L^{i+1} = L \cdot L^i$.

en général $L^i \neq \{w^i \mid w \in L\}$

miroir : $\mathcal{P}(\Sigma^*) \rightarrow \mathcal{P}(\Sigma^*)$, notée $\overleftarrow{L}$

$$\overleftarrow{L} = \{\overleftarrow{u} \mid u \in L\}$$

union, intersection, complémentation, différence, ... :

notés $L \cup L'$, $L \cap L'$, $\overline{L}$, $L \setminus L'$, ...

itération : $\mathcal{P}(\Sigma^*) \rightarrow \mathcal{P}(\Sigma^*)$, notée L^*

base : $\varepsilon \in L^*$; **règle** : si $u \in L$ et $v \in L^*$ alors $u \cdot v \in L^*$.

Rappels & notations : opérations sur les langages

concaténation : $\mathcal{P}(\Sigma^*) \times \mathcal{P}(\Sigma^*) \rightarrow \mathcal{P}(\Sigma^*)$, notée $L \cdot L'$

$$L \cdot L' = \{u \cdot v \mid u \in L, v \in L'\}$$

puissance : $\mathcal{P}(\Sigma^*) \times \mathbb{N} \rightarrow \mathcal{P}(\Sigma^*)$, notée L^i

base : $L^0 = \{\varepsilon\}$; **règle** : $L^{i+1} = L \cdot L^i$.

en général $L^i \neq \{w^i \mid w \in L\}$

miroir : $\mathcal{P}(\Sigma^*) \rightarrow \mathcal{P}(\Sigma^*)$, notée $\overleftarrow{L}$

$$\overleftarrow{L} = \{\overleftarrow{u} \mid u \in L\}$$

union, intersection, complémentation, différence, ... :

notés $L \cup L'$, $L \cap L'$, $\overline{L}$, $L \setminus L'$, ...

itération : $\mathcal{P}(\Sigma^*) \rightarrow \mathcal{P}(\Sigma^*)$, notée L^*

base : $\varepsilon \in L^*$; **règle** : si $u \in L$ et $v \in L^*$ alors $u \cdot v \in L^*$.

- Remarque: Σ^* est bien l'itération du langage Σ ,
le langage formé de tous les mots de longueur 1.

Rappels & notations : opérations sur les langages

concaténation : $\mathcal{P}(\Sigma^*) \times \mathcal{P}(\Sigma^*) \rightarrow \mathcal{P}(\Sigma^*)$, notée $L \cdot L'$

$$L \cdot L' = \{u \cdot v \mid u \in L, v \in L'\}$$

puissance : $\mathcal{P}(\Sigma^*) \times \mathbb{N} \rightarrow \mathcal{P}(\Sigma^*)$, notée L^i

$$\text{base : } L^0 = \{\varepsilon\}; \quad \text{r\`egle : } L^{i+1} = L \cdot L^i.$$

en g\'en\'eral $L^i \neq \{w^i \mid w \in L\}$

miroir : $\mathcal{P}(\Sigma^*) \rightarrow \mathcal{P}(\Sigma^*)$, not\'ee $\overleftarrow{L}$

$$\overleftarrow{L} = \{\overleftarrow{u} \mid u \in L\}$$

union, intersection, compl\'ementation, diff\'erence, ... :

not\'es $L \cup L'$, $L \cap L'$, $\overline{L}$, $L \setminus L'$, ...

it\'eration : $\mathcal{P}(\Sigma^*) \rightarrow \mathcal{P}(\Sigma^*)$, not\'ee L^*

$$\text{base : } \varepsilon \in L^*; \quad \text{r\`egle : si } u \in L \text{ et } v \in L^* \text{ alors } u \cdot v \in L^*.$$

it\'eration stricte : $\mathcal{P}(\Sigma^*) \rightarrow \mathcal{P}(\Sigma^*)$, not\'ee L^+

$$\text{base : } L \subseteq L^+; \quad \text{r\`egle : si } u \in L \text{ et } v \in L^+ \text{ alors } u \cdot v \in L^+.$$

- Remarque: Σ^* est bien l'it\'eration du langage Σ ,
le langage form\'e de tous les mots de longueur 1.

Exercices (opérations sur les langages)

- ▶ Que vaut : $L \cdot \emptyset$?
- ▶ Que valent : $\emptyset^0$? $\emptyset^3$? $\emptyset^*$? $\emptyset^+$?
- ▶ Montrer que la concaténation de langages est associative
et admet un élément neutre (quel est-il ?).
- ▶ Définir $L \cap L'$ sans utiliser l'opérateur $\cap$ d'intersection.
- ▶ Définir $L \cup L'$ sans utiliser l'opérateur $\cup$ d'union.
- ▶ Trouver L et i tels que $L^i \neq \{w^i \mid w \in L\}$.
- ▶ Montrer que $L^* = L^+ \cup \{\varepsilon\}$.
- ▶ Montrer que $L^* = \bigcup_{i \in \mathbb{N}} L^i$.
- ▶ Montrer que $L^+ = L \cdot L^*$.
- ▶ A-t-on toujours $L^+ = L^* \setminus \{\varepsilon\}$?

Exercices (opérations sur les langages)

- ▶ Que vaut : $L \cdot \emptyset$?
- ▶ Que valent : $\emptyset^0$? $\emptyset^3$? $\emptyset^*$? $\emptyset^+$?
- ▶ Montrer que la concaténation de langages est **associative**
et admet un **élément neutre** (quel est-il ?).
- ▶ Définir $L \cap L'$ sans utiliser l'opérateur $\cap$ d'intersection.
- ▶ Définir $L \cup L'$ sans utiliser l'opérateur $\cup$ d'union.
- ▶ Trouver L et i tels que $L^i \neq \{w^i \mid w \in L\}$.
- ▶ Montrer que $L^* = L^+ \cup \{\varepsilon\}$.
- ▶ Montrer que $L^* = \bigcup_{i \in \mathbb{N}} L^i$.
- ▶ Montrer que $L^+ = L \cdot L^*$.
- ▶ A-t-on toujours $L^+ = L^* \setminus \{\varepsilon\}$?

Rappels & notations : décompositions de mots

préfixe : u est préfixe de w s'il existe y tel que $w = uy$.

suffixe : u est suffixe de w s'il existe x tel que $w = xu$.

facteur : u est facteur de w s'il existe x et y tels que $w = xuy$.

Rappels & notations : décompositions de mots

préfixe : u est préfixe de w s'il existe y tel que $w = uy$.

suffixe : u est suffixe de w s'il existe x tel que $w = xu$.

facteur : u est facteur de w s'il existe x et y tels que $w = xuy$.

décomposition : une décomposition de w est une suite $w_1, \dots, w_n$ de mots telle que $w = w_1 \cdots w_n$.

Rappels & notations : décompositions de mots

préfixe : u est préfixe de w s'il existe y tel que $w = uy$.

suffixe : u est suffixe de w s'il existe x tel que $w = xu$.

facteur : u est facteur de w s'il existe x et y tels que $w = xuy$.

décomposition : une décomposition de w est une suite $w_1, \dots, w_n$ de mots telle que $w = w_1 \cdots w_n$.

sous-mot : u est un *sous-mot* de w s'il existe une décomposition $w_0 \cdots w_{2n}$ de w telle que $u = w_1 w_3 \cdots w_{2n-1}$

Rappels & notations : décompositions de mots

préfixe : u est préfixe de w s'il existe y tel que $w = uy$.

suffixe : u est suffixe de w s'il existe x tel que $w = xu$.

facteur : u est facteur de w s'il existe x et y tels que $w = xuy$.

décomposition : une décomposition de w est une suite $w_1, \dots, w_n$ de mots telle que $w = w_1 \cdots w_n$.

sous-mot : u est un *sous-mot* de w s'il existe une décomposition $w_0 \cdots w_{2n}$ de w telle que $u = u_1 u_3 \cdots u_{2n-1}$
e.g., *aa* est un sous-mot de *abbaba*

Rappels & notations : décompositions de mots

préfixe : u est préfixe de w s'il existe y tel que $w = uy$.

suffixe : u est suffixe de w s'il existe x tel que $w = xu$.

facteur : u est facteur de w s'il existe x et y tels que $w = xuy$.

décompositon : une décomposition de w est une suite $w_1, \dots, w_n$ de mots telle que $w = w_1 \cdots w_n$.

sous-mot : u est un *sous-mot* de w s'il existe une décomposition $w_0 \cdots w_{2n}$ de w telle que $u = u_1 u_3 \cdots u_{2n-1}$
e.g., *aa* est un sous-mot de *abbaba*

Remarques

- ▶ un facteur est un sous-mot.
- ▶ un préfixe/suffixe est un facteur.
- ▶ ε est préfixe/suffixe de tout mot.
- ▶ un mot est toujours préfixe/suffixe de lui-même.

Rapports & notations : décompositions de mots

préfixe : u est préfixe de w s'il existe y tel que $w = uy$.

suffixe : u est suffixe de w s'il existe x tel que $w = xu$.

facteur : u est facteur de w s'il existe x et y tels que $w = xuy$.

décomposition : une décomposition de w est une suite $w_1, \dots, w_n$ de mots telle que $w = w_1 \cdots w_n$.

sous-mot : u est un *sous-mot* de w s'il existe une décomposition $w_0 \cdots w_{2n}$ de w telle que $u = u_1 u_3 \cdots u_{2n-1}$
e.g., *aa* est un sous-mot de *abbaba*

Remarques

- ▶ un facteur est un sous-mot.
- ▶ un préfixe/suffixe est un facteur.
- ▶ ε est préfixe/suffixe de tout mot.
- ▶ un mot est toujours préfixe/suffixe de lui-même.

facteur propre : un facteur qui n'est pas ε (préfixe/suffixe propre)

facteur strict : un facteur qui n'est pas le mot (préfixe/suffixe strict)

Descriptions de langages

par extension : e.g., $\{a, abb, baba\}$

Descriptions de langages

par extension : e.g., $\{a, abb, baba\}$

par compréhension : e.g., $\{b, ba, baa, baaa, \dots\}$

Descriptions de langages

par extension : e.g., $\{a, abb, baba\}$

par compréhension : e.g., $\{b, ba, baa, baaa, \dots\}$

par intention : e.g., $\{w \in \Sigma^* \mid |w|_b = 1 \text{ et } b \text{ préfixe de } w\}$

Descriptions de langages

par extension : e.g., $\{a, abb, baba\}$

par compréhension : e.g., $\{b, ba, baa, baaa, \dots\}$

par intention : e.g., $\{w \in \Sigma^* \mid |w|_b = 1 \text{ et } b \text{ préfixe de } w\}$

par induction : e.g., **base** : $b \in L$ **règle** : si $u \in L$, alors $u \cdot a \in L$

Descriptions de langages

par extension : e.g., $\{a, abb, baba\}$

par compréhension : e.g., $\{b, ba, baa, baaa, \dots\}$

par intention : e.g., $\{w \in \Sigma^* \mid |w|_b = 1 \text{ et } b \text{ préfixe de } w\}$

par induction : e.g., **base** : $b \in L$ **règle** : si $u \in L$, alors $u \cdot a \in L$

Et à partir d'autres langages :

par opération : e.g., $L' \cdot (\overline{L} \cap L')$

Descriptions de langages

par extension : e.g., $\{a, abb, baba\}$

par compréhension : e.g., $\{b, ba, baa, baaa, \dots\}$

par intention : e.g., $\{w \in \Sigma^* \mid |w|_b = 1 \text{ et } b \text{ préfixe de } w\}$

par induction : e.g., **base** : $b \in L$ **règle** : si $u \in L$, alors $u \cdot a \in L$

Et à partir d'autres langages :

par opération : e.g., $L' \cdot (\bar{L} \cap L')$, *expressions régulières* (e.g., $a(ba+ab)^*b$).

Descriptions de langages

par extension : e.g., $\{a, abb, baba\}$

par compréhension : e.g., $\{b, ba, baa, baaa, \dots\}$

par intention : e.g., $\{w \in \Sigma^* \mid |w|_b = 1 \text{ et } b \text{ préfixe de } w\}$

par induction : e.g., **base** : $b \in L$ **règle** : si $u \in L$, alors $u \cdot a \in L$

Et à partir d'autres langages :

par opération : e.g., $L' \cdot (\bar{L} \cap L')$, *expressions régulières* (e.g., $a(ba+ab)^*b$).

■ **Question** : Quels langages peut-on décrire :

- ▶ par extension ?
- ▶ par opération à partir des langages décrits par extension ?

Descriptions de langages

par extension : e.g., $\{a, abb, baba\}$

par compréhension : e.g., $\{b, ba, baa, baaa, \dots\}$

par intention : e.g., $\{w \in \Sigma^* \mid |w|_b = 1 \text{ et } b \text{ préfixe de } w\}$

par induction : e.g., **base** : $b \in L$ **règle** : si $u \in L$, alors $u \cdot a \in L$

Et à partir d'autres langages :

par opération : e.g., $L' \cdot (\bar{L} \cap L')$, *expressions régulières* (e.g., $a(ba+ab)^*b$).

■ **Question** : Quels langages peut-on décrire :

▶ par extension ?

Les langages finis.

▶ par opération à partir des langages décrits par extension ?

Descriptions de langages

par extension : e.g., $\{a, abb, baba\}$

par compréhension : e.g., $\{b, ba, baa, baaa, \dots\}$

par intention : e.g., $\{w \in \Sigma^* \mid |w|_b = 1 \text{ et } b \text{ préfixe de } w\}$

par induction : e.g., **base** : $b \in L$ **règle** : si $u \in L$, alors $u \cdot a \in L$

Et à partir d'autres langages :

par opération : e.g., $L' \cdot (\bar{L} \cap L')$, *expressions régulières* (e.g., $a(ba+ab)^*b$).

■ **Question** : Quels langages peut-on décrire :

▶ par extension ?

Les langages finis.

▶ par opération à partir des langages décrits par extension ?

Les langages *réguliers* (si opérations précédentes).

Descriptions de langages

par extension : e.g., $\{a, abb, baba\}$

par compréhension : e.g., $\{b, ba, baa, baaa, \dots\}$

par intention : e.g., $\{w \in \Sigma^* \mid |w|_b = 1 \text{ et } b \text{ préfixe de } w\}$

par induction : e.g., **base** : $b \in L$ **règle** : si $u \in L$, alors $u \cdot a \in L$

par modèle accepteur : e.g., un *automate*, un programme, une fonction
le langage *reconnu* est l'ensemble des mots *acceptés*.

Et à partir d'autres langages :

par opération : e.g., $L' \cdot (\bar{L} \cap L')$, *expressions régulières* (e.g., $a(ba+ab)^*b$).

■ **Question** : Quels langages peut-on décrire :

► par extension ?

Les langages finis.

► par opération à partir des langages décrits par extension ?

Les langages *réguliers* (si opérations précédentes).

Descriptions de langages

par extension : e.g., $\{a, abb, baba\}$

par compréhension : e.g., $\{b, ba, baa, baaa, \dots\}$

par intention : e.g., $\{w \in \Sigma^* \mid |w|_b = 1 \text{ et } b \text{ préfixe de } w\}$

par induction : e.g., **base** : $b \in L$ **règle** : si $u \in L$, alors $u \cdot a \in L$

par modèle accepteur : e.g., un *automate*, un programme, une fonction
le langage *reconnu* est l'ensemble des mots *acceptés*.

Et à partir d'autres langages :

par opération : e.g., $L' \cdot (\bar{L} \cap L')$, *expressions régulières* (e.g., $a(ba+ab)^*b$).

■ **Question** : Quels langages peut-on décrire :

► par extension ?

Les langages finis.

► par opération à partir des langages décrits par extension ?

Les langages *réguliers* (si opérations précédentes).

Quel modèle pour les langages *réguliers* ?

Descriptions de langages

par extension : e.g., $\{a, abb, baba\}$

par compréhension : e.g., $\{b, ba, baa, baaa, \dots\}$

par intention : e.g., $\{w \in \Sigma^* \mid |w|_b = 1 \text{ et } b \text{ préfixe de } w\}$

par induction : e.g., **base** : $b \in L$ **règle** : si $u \in L$, alors $u \cdot a \in L$

par modèle accepteur : e.g., un *automate*, un programme, une fonction
le langage *reconnu* est l'ensemble des mots *acceptés*.

Et à partir d'autres langages :

par opération : e.g., $L' \cdot (\bar{L} \cap L')$, *expressions régulières* (e.g., $a(ba+ab)^*b$).

■ **Question** : Quels langages peut-on décrire :

► par extension ?

Les langages finis.

► par opération à partir des langages décrits par extension ?

Les langages *réguliers* (si opérations précédentes).

Quel modèle pour les langages *réguliers* ?

Les automates finis.

Descriptions de langages

par extension : e.g., $\{a, abb, baba\}$

par compréhension : e.g., $\{b, ba, baa, baaa, \dots\}$

par intention : e.g., $\{w \in \Sigma^* \mid |w|_b = 1 \text{ et } b \text{ préfixe de } w\}$

par induction : e.g., **base** : $b \in L$ **règle** : si $u \in L$, alors $u \cdot a \in L$

par modèle accepteur : e.g., un *automate*, un programme, une fonction
le langage *reconnu* est l'ensemble des mots *acceptés*.

par un modèle générateur : e.g., les *grammaires*, les *inductions croisées*.

Et à partir d'autres langages :

par opération : e.g., $L' \cdot (\bar{L} \cap L')$, *expressions régulières* (e.g., $a(ba+ab)^*b$).

■ **Question** : Quels langages peut-on décrire :

▶ par extension ?

Les langages finis.

▶ par opération à partir des langages décrits par extension ?

Les langages *réguliers* (si opérations précédentes).

Quel modèle pour les langages *réguliers* ?

Les automates finis.

Rappels & notations : automates finis

Un *automate fini* est un quintuplet : $\langle Q, \Sigma, I, F, \delta \rangle$ où :

- ▶ Q est un ensemble fini ; $q \in Q$ est appelé *états* ;
- ▶ Σ est l'alphabet d'entrée ;
- ▶ $I \subseteq Q$; $q \in I$ est dit *initial* ;
- ▶ $F \subseteq Q$; $q \in F$ est dit *final* ou *acceptant* ;
- ▶ $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ appelée *fonction de transition*.

Rappels & notations : automates finis

Un *automate fini* est un quintuplet : $\langle Q, \Sigma, I, F, \delta \rangle$ où :

- ▶ Q est un ensemble fini ; $q \in Q$ est appelé *états* ;
- ▶ Σ est l'alphabet d'entrée ;
- ▶ $I \subseteq Q$; $q \in I$ est dit *initial* ;
- ▶ $F \subseteq Q$; $q \in F$ est dit *final* ou *acceptant* ;
- ▶ $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ appelée *fonction de transition*.

Un automate est dit :

déterministe : si $|I| = 1$ et $\forall p \in Q \forall \sigma \in \Sigma, |\delta(p, \sigma)| \leq 1$;

Rappels & notations : automates finis

Un *automate fini* est un quintuplet : $\langle Q, \Sigma, I, F, \delta \rangle$ où :

- ▶ Q est un ensemble fini ; $q \in Q$ est appelé *états* ;
- ▶ Σ est l'alphabet d'entrée ;
- ▶ $I \subseteq Q$; $q \in I$ est dit *initial* ;
- ▶ $F \subseteq Q$; $q \in F$ est dit *final* ou *acceptant* ;
- ▶ $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ appelée *fonction de transition*.

Un automate est dit :

déterministe : si $|I| = 1$ et $\forall p \in Q \forall \sigma \in \Sigma, |\delta(p, \sigma)| \leq 1$;

on supposera alors abusivement $\delta : Q \times \Sigma \rightarrow Q$

Rappels & notations : automates finis

Un *automate fini* est un quintuplet : $\langle Q, \Sigma, I, F, \delta \rangle$ où :

- ▶ Q est un ensemble fini ; $q \in Q$ est appelé *états* ;
- ▶ Σ est l'alphabet d'entrée ;
- ▶ $I \subseteq Q$; $q \in I$ est dit *initial* ;
- ▶ $F \subseteq Q$; $q \in F$ est dit *final* ou *acceptant* ;
- ▶ $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ appelée *fonction de transition*.

Un automate est dit :

déterministe : si $|I| = 1$ et $\forall p \in Q \forall \sigma \in \Sigma, |\delta(p, \sigma)| \leq 1$;

on supposera alors abusivement $\delta : Q \times \Sigma \rightarrow Q$

total : si $\forall p \in Q \forall \sigma \in \Sigma, |\delta(p, \sigma)| \geq 1$;

Rappels & notations : automates finis

Un *automate fini* est un quintuplet : $\langle Q, \Sigma, I, F, \delta \rangle$ où :

- ▶ Q est un ensemble fini ; $q \in Q$ est appelé *états* ;
- ▶ Σ est l'alphabet d'entrée ;
- ▶ $I \subseteq Q$; $q \in I$ est dit *initial* ;
- ▶ $F \subseteq Q$; $q \in F$ est dit *final* ou *acceptant* ;
- ▶ $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ appelée *fonction de transition*.

Un automate est dit :

déterministe : si $|I| = 1$ et $\forall p \in Q \forall \sigma \in \Sigma, |\delta(p, \sigma)| \leq 1$;

on supposera alors abusivement $\delta : Q \times \Sigma \rightarrow Q$

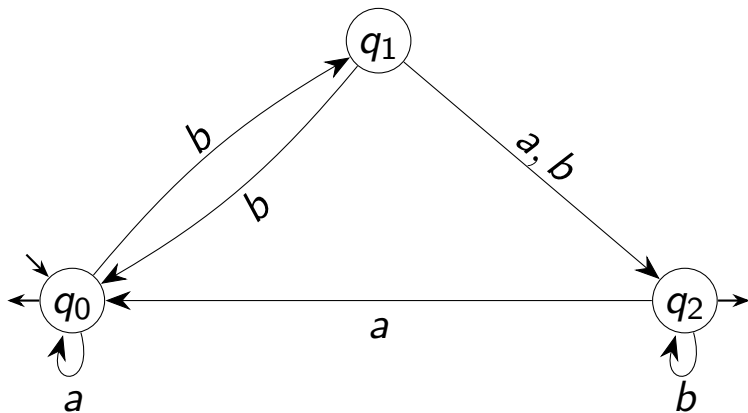
total : si $\forall p \in Q \forall \sigma \in \Sigma, |\delta(p, \sigma)| \geq 1$;

Un automate peut être donné :

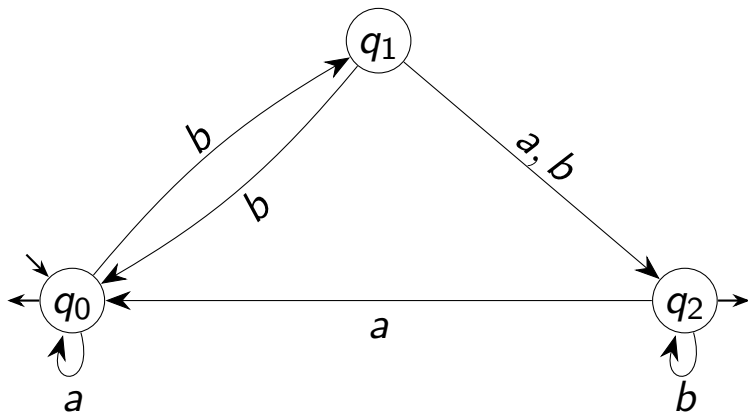
- ▶ graphiquement
 graphe orienté avec arête étiquetées par des (ensembles de) lettres ;
- ▶ par sa table de transition

matrice $Q \times \Sigma$ avec coefficient dans $\mathcal{P}(Q)$.

Un automate fini



Un automate fini

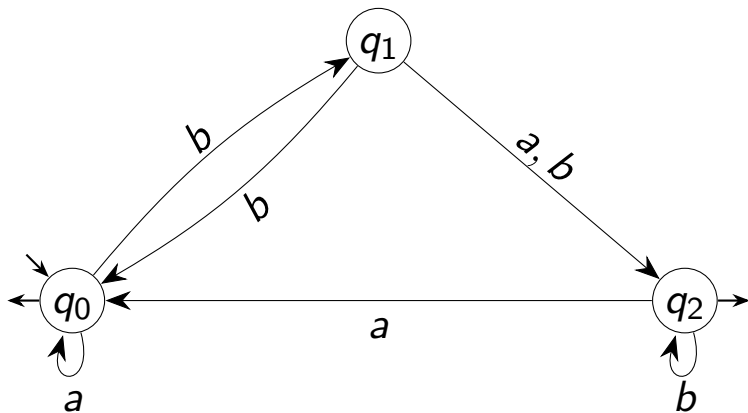


Quels sont ses états initiaux ? ses états finaux ?

Accepte-t-il *bab* ? *abbb* ? *baab* ? *bbaab* ?

Est-il total ? déterministe ?

Un automate fini



Quels sont ses états initiaux ? $I = \{q_0\}$. ses états finaux ? $F = \{q_0, q_2\}$.
Accepte-t-il *bab* ? **oui**. *abbb* ? **oui**. *baab* ? **non**. *bbaab* ? **non**.
Est-il total ? **oui**. déterministe ? **non**.

Généralisation de la fonction de transition

On peut étendre δ :

Généralisation de la fonction de transition

On peut étendre δ :

aux mots : $\hat{\delta} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$

Généralisation de la fonction de transition

On peut étendre δ :

aux mots : $\hat{\delta} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$

base : $\hat{\delta}(p, \varepsilon) = \{p\}$; **règle** : $\hat{\delta}(p, a \cdot u) = \bigcup_{q \in \delta(p, a)} \hat{\delta}(q, u)$.

Généralisation de la fonction de transition

On peut étendre δ :

aux mots : $\hat{\delta} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$

base : $\hat{\delta}(p, \varepsilon) = \{p\}$; **règle :** $\hat{\delta}(p, a \cdot u) = \bigcup_{q \in \delta(p, a)} \hat{\delta}(q, u).$

$q \in \hat{\delta}(p, abbab)$ si depuis p on peut atteindre q en lisant *abbab*.

Généralisation de la fonction de transition

On peut étendre δ :

aux mots : $\hat{\delta} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$

base : $\hat{\delta}(p, \varepsilon) = \{p\}$; **règle** : $\hat{\delta}(p, a \cdot u) = \bigcup_{q \in \delta(p, a)} \hat{\delta}(q, u).$

$q \in \hat{\delta}(p, abbab)$ si depuis p on peut atteindre q en lisant *abbab*.

aux ensembles d'états : $\tilde{\delta} : \mathcal{P}(Q) \times \Sigma \rightarrow \mathcal{P}(Q) :$

Généralisation de la fonction de transition

On peut étendre δ :

aux mots : $\hat{\delta} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$

base : $\hat{\delta}(p, \varepsilon) = \{p\}$; **règle** : $\hat{\delta}(p, a \cdot u) = \bigcup_{q \in \delta(p, a)} \hat{\delta}(q, u).$

$q \in \hat{\delta}(p, abbab)$ si depuis p on peut atteindre q en lisant *abbab*.

aux ensembles d'états : $\tilde{\delta} : \mathcal{P}(Q) \times \Sigma \rightarrow \mathcal{P}(Q) : \quad \tilde{\delta}(X, a) = \bigcup_{q \in X} \delta(q, a).$

Généralisation de la fonction de transition

On peut étendre δ :

aux mots : $\hat{\delta} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$

base : $\hat{\delta}(p, \varepsilon) = \{p\}$; **règle** : $\hat{\delta}(p, a \cdot u) = \bigcup_{q \in \delta(p, a)} \hat{\delta}(q, u).$

$q \in \hat{\delta}(p, abbab)$ si depuis p on peut atteindre q en lisant *abbab*.

aux ensembles d'états : $\tilde{\delta} : \mathcal{P}(Q) \times \Sigma \rightarrow \mathcal{P}(Q) : \quad \tilde{\delta}(X, a) = \bigcup_{q \in X} \delta(q, a).$

$q \in \tilde{\delta}(X, a)$ si depuis un état de X on peut atteindre q en lisant a .

Généralisation de la fonction de transition

On peut étendre δ :

aux mots : $\hat{\delta} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$

base : $\hat{\delta}(p, \varepsilon) = \{p\}$; **règle** : $\hat{\delta}(p, a \cdot u) = \bigcup_{q \in \delta(p, a)} \hat{\delta}(q, u).$

$q \in \hat{\delta}(p, abbab)$ si depuis p on peut atteindre q en lisant *abbab*.

aux ensembles d'états : $\tilde{\delta} : \mathcal{P}(Q) \times \Sigma \rightarrow \mathcal{P}(Q) : \quad \tilde{\delta}(X, a) = \bigcup_{q \in X} \delta(q, a).$

$q \in \tilde{\delta}(X, a)$ si depuis un état de X on peut atteindre q en lisant a .

les deux : $\tilde{\tilde{\delta}} : \mathcal{P}(Q) \times \Sigma^* \rightarrow \mathcal{P}(Q)$

Généralisation de la fonction de transition

On peut étendre δ :

aux mots : $\widehat{\delta} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$

base : $\widehat{\delta}(p, \varepsilon) = \{p\}$; **règle** : $\widehat{\delta}(p, a \cdot u) = \bigcup_{q \in \delta(p, a)} \widehat{\delta}(q, u).$

$q \in \widehat{\delta}(p, abbab)$ si depuis p on peut atteindre q en lisant *abbab*.

aux ensembles d'états : $\widetilde{\delta} : \mathcal{P}(Q) \times \Sigma \rightarrow \mathcal{P}(Q) : \quad \widetilde{\delta}(X, a) = \bigcup_{q \in X} \delta(q, a).$

$q \in \widetilde{\delta}(X, a)$ si depuis un état de X on peut atteindre q en lisant a .

les deux : $\widetilde{\widehat{\delta}} : \mathcal{P}(Q) \times \Sigma^* \rightarrow \mathcal{P}(Q)$

$\widetilde{\widehat{\delta}}(X, u) = \bigcup_{q \in X} \widehat{\delta}(q, u).$

Généralisation de la fonction de transition

On peut étendre δ :

aux mots : $\widehat{\delta} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$

base : $\widehat{\delta}(p, \varepsilon) = \{p\}$; **règle** : $\widehat{\delta}(p, a \cdot u) = \bigcup_{q \in \delta(p, a)} \widehat{\delta}(q, u).$

$q \in \widehat{\delta}(p, \text{abbab})$ si depuis p on peut atteindre q en lisant *abbab*.

aux ensembles d'états : $\widetilde{\delta} : \mathcal{P}(Q) \times \Sigma \rightarrow \mathcal{P}(Q) : \quad \widetilde{\delta}(X, a) = \bigcup_{q \in X} \delta(q, a).$

$q \in \widetilde{\delta}(X, a)$ si depuis un état de X on peut atteindre q en lisant a .

les deux : $\widetilde{\widetilde{\delta}} : \mathcal{P}(Q) \times \Sigma^* \rightarrow \mathcal{P}(Q)$

$\widetilde{\widetilde{\delta}}(X, u) = \bigcup_{q \in X} \widehat{\delta}(q, u).$

$q \in \widetilde{\widetilde{\delta}}(X, \text{abbab})$ si $\exists x \in X$, depuis x on peut atteindre q en lisant *abbab*.

Généralisation de la fonction de transition

On peut étendre δ :

aux mots : $\hat{\delta} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$

base : $\hat{\delta}(p, \varepsilon) = \{p\}$; **règle** : $\hat{\delta}(p, a \cdot u) = \bigcup_{q \in \delta(p, a)} \hat{\delta}(q, u).$

$q \in \hat{\delta}(p, \text{abbab})$ si depuis p on peut atteindre q en lisant *abbab*.

aux ensembles d'états : $\tilde{\delta} : \mathcal{P}(Q) \times \Sigma \rightarrow \mathcal{P}(Q) : \tilde{\delta}(X, a) = \bigcup_{q \in X} \delta(q, a).$

$q \in \tilde{\delta}(X, a)$ si depuis un état de X on peut atteindre q en lisant a .

les deux : $\tilde{\tilde{\delta}} : \mathcal{P}(Q) \times \Sigma^* \rightarrow \mathcal{P}(Q)$

$\tilde{\tilde{\delta}}(X, u) = \bigcup_{q \in X} \hat{\delta}(q, u).$

$q \in \tilde{\tilde{\delta}}(X, \text{abbab})$ si $\exists x \in X$, depuis x on peut atteindre q en lisant *abbab*.

On notera abusivement δ pour chacune de ces fonctions

(on a $\delta(q, u) = \delta(\{q\}, u)$).

Acceptation

Un mot u est *accepté* par A si et seulement si

Acceptation

Un mot u est *accepté* par A si et seulement si
il existe $i \in I$ et $f \in F$ tels que $f \in \delta(i, u)$

Acceptation

Un mot u est *accepté* par A si et seulement si $\delta(I, u) \cap F \neq \emptyset$, i.e.,
il existe $i \in I$ et $f \in F$ tels que $f \in \delta(i, u)$

Acceptation

Un mot u est *accepté* par A si et seulement si $\delta(I, u) \cap F \neq \emptyset$, i.e.,
il existe $i \in I$ et $f \in F$ tels que $f \in \delta(i, u)$
sinon, on dit qu'il est *rejeté*.

Acceptation

Un mot u est *accepté* par A si et seulement si $\delta(I, u) \cap F \neq \emptyset$, i.e.,
il existe $i \in I$ et $f \in F$ tels que $f \in \delta(i, u)$
sinon, on dit qu'il est *rejeté*.

Le langage *reconnu* par A est l'ensemble des mots acceptés par A

Acceptation

Un mot u est *accepté* par A si et seulement si $\delta(I, u) \cap F \neq \emptyset$, i.e.,
il existe $i \in I$ et $f \in F$ tels que $f \in \delta(i, u)$
sinon, on dit qu'il est *rejeté*.

Le langage *reconnu* par A est l'ensemble des mots acceptés par A ;
on le notera $\mathcal{L}(A)$.

Acceptation

Un mot u est *accepté* par A si et seulement si $\delta(I, u) \cap F \neq \emptyset$, i.e.,
il existe $i \in I$ et $f \in F$ tels que $f \in \delta(i, u)$
sinon, on dit qu'il est *rejeté*.

Le langage *reconnu* par A est l'ensemble des mots acceptés par A ;
on le notera $\mathcal{L}(A)$.

Deux automates sont *équivalents* s'ils reconnaissent le même langage.

Acceptation

Un mot u est *accepté* par A si et seulement si $\delta(I, u) \cap F \neq \emptyset$, i.e.,
il existe $i \in I$ et $f \in F$ tels que $f \in \delta(i, u)$
sinon, on dit qu'il est *rejeté*.

Le langage *reconnu* par A est l'ensemble des mots acceptés par A ;
on le notera $\mathcal{L}(A)$.

Deux automates sont *équivalents* s'ils reconnaissent le même langage.

Un *calcul de A sur u* est une suite $p_0, p_1, \dots, p_n$ d'états
telle que $p_{i+1} \in \delta(p_i, u[i])$ où $u[i]$ est la $(i+1)^{\text{e}}$ lettre de u .

Acceptation

Un mot u est *accepté* par A si et seulement si $\delta(I, u) \cap F \neq \emptyset$, i.e.,
il existe $i \in I$ et $f \in F$ tels que $f \in \delta(i, u)$
sinon, on dit qu'il est *rejeté*.

Le langage *reconnu* par A est l'ensemble des mots acceptés par A ;
on le notera $\mathcal{L}(A)$.

Deux automates sont *équivalents* s'ils reconnaissent le même langage.

Un *calcul de A sur u* est une suite $p_0, p_1, \dots, p_n$ d'états
telle que $p_{i+1} \in \delta(p_i, u[i])$ où $u[i]$ est la $(i+1)^{\text{e}}$ lettre de u .

Un calcul $p_0, \dots, p_n$ est :

- ▶ *avorté* si $n < |u|$ et $\delta(p_n, u[n]) = \emptyset$;
- ▶ *initial* si $p_0 \in I$;
- ▶ *final* (ou *acceptant*) si $p_n \in F$ et $n = |u|$;
- ▶ *réussi* s'il est à la fois initial et final.

Acceptation

Un mot u est *accepté* par A si et seulement si $\delta(I, u) \cap F \neq \emptyset$, i.e.,
il existe $i \in I$ et $f \in F$ tels que $f \in \delta(i, u)$
sinon, on dit qu'il est *rejeté*.

Le langage *reconnu* par A est l'ensemble des mots acceptés par A ;
on le notera $\mathcal{L}(A)$.

Deux automates sont *équivalents* s'ils reconnaissent le même langage.

Un *calcul de A sur u* est une suite $p_0, p_1, \dots, p_n$ d'états
telle que $p_{i+1} \in \delta(p_i, u[i])$ où $u[i]$ est la $(i+1)^{\text{e}}$ lettre de u .

Un calcul $p_0, \dots, p_n$ est :

- ▶ *avorté* si $n < |u|$ et $\delta(p_n, u[n]) = \emptyset$;
- ▶ *initial* si $p_0 \in I$;
- ▶ *final* (ou *acceptant*) si $p_n \in F$ et $n = |u|$;
- ▶ *réussi* s'il est à la fois initial et final.

■ Remarque: u est accepté par A ssi il existe un calcul réussi de A sur u .

Rappels & notations : Transformations d'automates

Soit un automate :

A reconnaissant $L = \mathcal{L}(A)$

Rappels & notations : Transformations d'automates

Soit un automate :

A reconnaissant $L = \mathcal{L}(A)$

Vous savez construire un automate équivalent à A :

- ▶ total ;
- ▶ déterministe ;
- ▶ total & déterministe ;
- ▶ déterministe *minimal* (?).

Rappels & notations : Transformations d'automates

Soit un automate :

A reconnaissant $L = \mathcal{L}(A)$

Vous savez construire un automate équivalent à A :

- ▶ total ;
- ▶ déterministe ;
- ▶ total & déterministe ;
- ▶ déterministe *minimal* (?).

ou bien, où tous les états sont :

- ▶ *accessibles* ;
- ▶ *co-accessibles* ;
- ▶ *utiles* (i.e., les 2).

Rappels & notations : Transformations d'automates

Soient deux automates :

A reconnaissant $L = \mathcal{L}(A)$
et A' reconnaissant $L' = \mathcal{L}(A')$.

Vous savez construire un automate équivalent à A :

- ▶ total ;
- ▶ déterministe ;
- ▶ total & déterministe ;
- ▶ déterministe *minimal* (?).

ou bien, où tous les états sont :

- ▶ *accessibles* ;
- ▶ *co-accessibles* ;
- ▶ *utiles* (i.e., les 2).

Vous savez construire un automate reconnaissant :

Rappels & notations : Transformations d'automates

Soient deux automates :

A reconnaissant $L = \mathcal{L}(A)$
et A' reconnaissant $L' = \mathcal{L}(A')$.

Vous savez construire un automate équivalent à A :

- ▶ total ;
- ▶ déterministe ;
- ▶ total & déterministe ;
- ▶ déterministe *minimal* (?).

ou bien, où tous les états sont :

- ▶ *accessibles* ;
- ▶ *co-accessibles* ;
- ▶ *utiles* (i.e., les 2).

Vous savez construire un automate reconnaissant :

- ▶ $L \cup L'$;
- ▶ $L \cap L'$;
- ▶ $L \setminus L'$;
- ▶ $L \nabla L'$;
- ▶ $L \cdot L'$;
- ▶ $L^i, i \in \mathbb{N}$;
- ▶ $\bar{L} = \Sigma^* \setminus L$;
- ▶ L^* ;
- ▶ L^+ ;
- ▶ $\overleftarrow{L}$;
- ▶ $\text{Pref}(L)$;
- ▶ $\text{Suff}(L)$;
- ▶ $\text{Fact}(L)$;
- ▶ $\text{Sub}(L)$.

Rappels & notations : Transformations d'automates

Soient deux automates :

A reconnaissant $L = \mathcal{L}(A)$
et A' reconnaissant $L' = \mathcal{L}(A')$.

Vous savez construire un automate équivalent à A :

- ▶ total ;
- ▶ déterministe ;
- ▶ **total & déterministe** ;
- ▶ déterministe *minimal* (?).

ou bien, où tous les états sont :

- ▶ *accessibles* ;
- ▶ *co-accessibles* ;
- ▶ *utiles* (i.e., les 2).

Vous savez construire un automate reconnaissant :

- ▶ $L \cup L'$;
- ▶ $L \cap L'$;
- ▶ $L \setminus L'$;
- ▶ $L \nabla L'$;
- ▶ $L \cdot L'$;
- ▶ $L^i, i \in \mathbb{N}$;
- ▶ $\overline{L} = \Sigma^* \setminus L$;
- ▶ L^* ;
- ▶ L^+ ;
- ▶ $\overleftarrow{L}$;
- ▶ $\text{Pref}(L)$;
- ▶ $\text{Suff}(L)$;
- ▶ $\text{Fact}(L)$;
- ▶ $\text{Sub}(L)$.

Rappels & notation : expression régulière

Les expressions régulières sont définies inductivement par :

bases : sont des expressions régulières :

- ▶ $\emptyset$
- ▶ a , pour chaque $a \in \Sigma$;

règles : si e et f sont des expressions régulières, alors aussi :

- ▶ $e + f$
- ▶ $e \cdot f$ (ou ef)
- ▶ e^*

Rappels & notation : expression régulière

Les expressions régulières sont définies inductivement par :

bases : sont des expressions régulières :

- ▶ $\emptyset$
- ▶ a , pour chaque $a \in \Sigma$;

règles : si e et f sont des expressions régulières, alors aussi :

- ▶ $e + f$
- ▶ $e \cdot f$ (ou ef)
- ▶ e^*

On suppose les priorités naturelles des opérateurs : $*$ $>$ $\cdot$ $>$ $+$
et on utilise le parenthésage si besoin.

Rappels & notation : expression régulière

Les expressions régulières sont définies inductivement par :

bases : sont des expressions régulières :

- ▶ $\emptyset$
- ▶ a , pour chaque $a \in \Sigma$;

règles : si e et f sont des expressions régulières, alors aussi :

- ▶ $e + f$
- ▶ $e \cdot f$ (ou ef)
- ▶ e^*

On suppose les priorités naturelles des opérateurs : $*$ $>$ $\cdot$ $>$ $+$
et on utilise le parenthésage si besoin.

On s'autorise les raccourcis suivants :

- ▶ $\varepsilon = \emptyset^*$;
- ▶ $e^+ = ee^*$;
- ▶ $e^i = \underbrace{e \cdot e \cdots e}_{i \text{ fois}}, \text{ pour } i \in \mathbb{N}.$

Rappels & notations : Théorème de Kleene

Théorème (Kleene)

Les langages décrits par des expressions régulières
sont exactement les langages reconnus par des automates .

Théorème (Kleene)

Les langages décrits par des expressions régulières (Reg)
sont exactement les langages reconnus par des automates (Rec).

Démonstration:

- ▶ $\text{Reg} \subseteq \text{Rec}$: conséquence de la slide 13.
- ▶ $\text{Rec} \subseteq \text{Reg}$: algorithme de Brzozowski et McCluskey (par exemple).



Rappels & notations : Théorème de Kleene

Théorème (Kleene)

Les langages décrits par des expressions régulières (Reg)
sont exactement les langages reconnus par des automates (Rec).

Démonstration:

- ▶ $\text{Reg} \subseteq \text{Rec}$: conséquence de la slide 13.
- ▶ $\text{Rec} \subseteq \text{Reg}$: algorithme de Brzozowski et McCluskey (par exemple).



Corollaire:

- ▶ Reg est clos par intersection, complémentation et miroir.
- ▶ On peut tester l'équivalence de deux expressions régulières.

Frontière des réguliers

Il existe des langages non-réguliers.

Frontière des réguliers

Il existe des langages non-réguliers.

- Rappels: pour montrer qu'un langage n'est pas régulier, on peut utiliser :
 - ▶ le lemme de pompage
 - ▶ les propriétés de clôture
 - ▶ les congruences de Myhill-Nerode (classes d'équivalence ; pas vues)

Frontière des réguliers

Il existe des langages non-réguliers.

- Rappels: pour montrer qu'un langage n'est pas régulier, on peut utiliser :
 - ▶ le lemme de pompage
 - ▶ les propriétés de clôture
 - ▶ les congruences de Myhill-Nerode (classes d'équivalence ; pas vues)

Exercices

Montrer que les langages suivants ne sont pas réguliers :

- ▶ le langage $\{a^n b^n \mid n \in \mathbb{N}\}$;
- ▶ le langage des mots bien parenthésés ;
- ▶ le langage des mots qui ont autant de a que de b ;
- ▶ le langage $\{a^n b^m c^k \mid n \neq m \text{ ou } m \neq k\}$;
- ▶ les mots dont la longueur est une puissance de 2.